

LOB Price Prediction via Knowledge Distillation

Adrian Walczak

Michał Wilczkowski

Wiktor Zoga

Problem definition

- **Core Objective**

- Predict the next mid-price direction (Up, Flat, or Down) over a specific forward horizon.
- *Formula*: $\text{Mid-Price} = (\text{Best Ask Price} + \text{Best Bid Price}) / 2$

- **Feature Space**

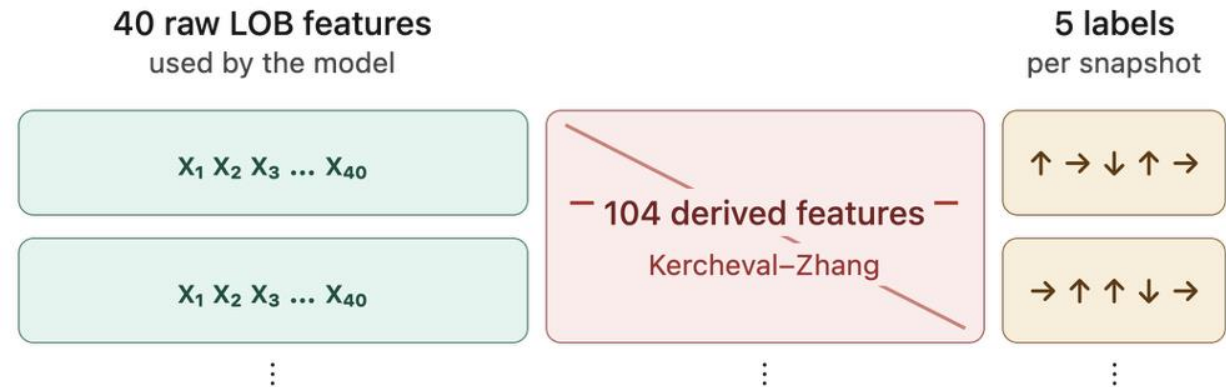
- High-frequency sequential grid of size: 50 Ticks x 40 Features.
- Looks back at a rolling window of the last 50 consecutive market updates.
- Features track 10 structural layers of the Limit Order Book (Prices and Volumes for Bid/Ask sides).

- **Target Classification**

- **Class 0 (Up)**: Mid-Price moves up by more than some threshold.
- **Class 1 (Flat)**: Mid-Price stays stationary within some threshold.
- **Class 2 (Down)**: Mid-Price moves down by more than some threshold.

Dataset: FI-2010

- First publicly available benchmark dataset for LOB mid-price forecasting [1]
- 5 Finnish stocks
- Each snapshot contains:
 - **40 raw LOB features**
 - 104 derived features (unused)
 - **5 labels** (one per horizon)



LOB features

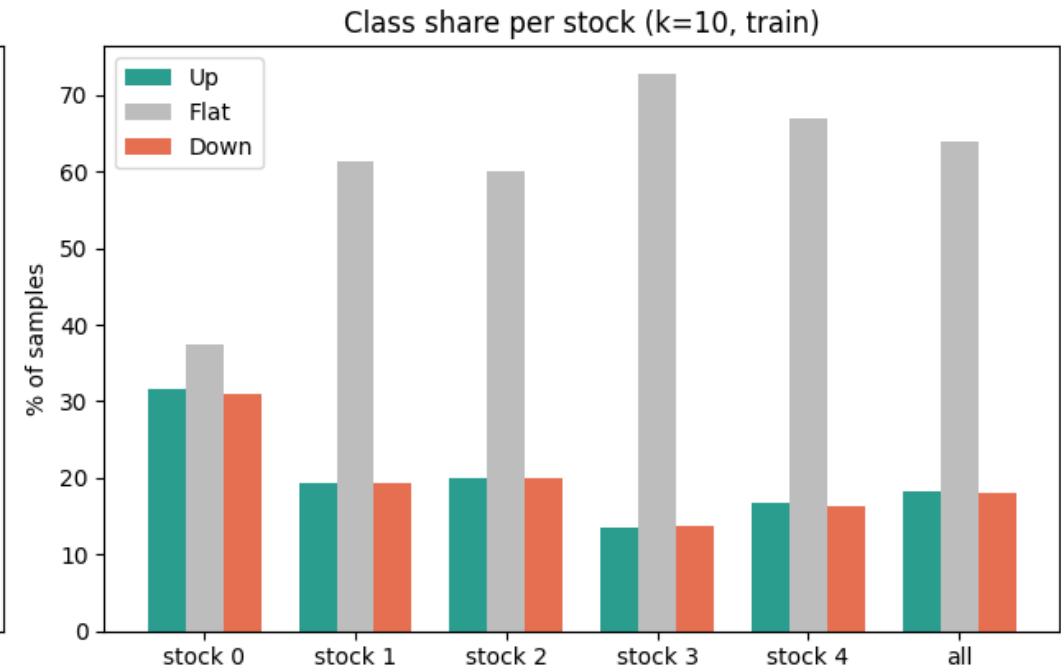
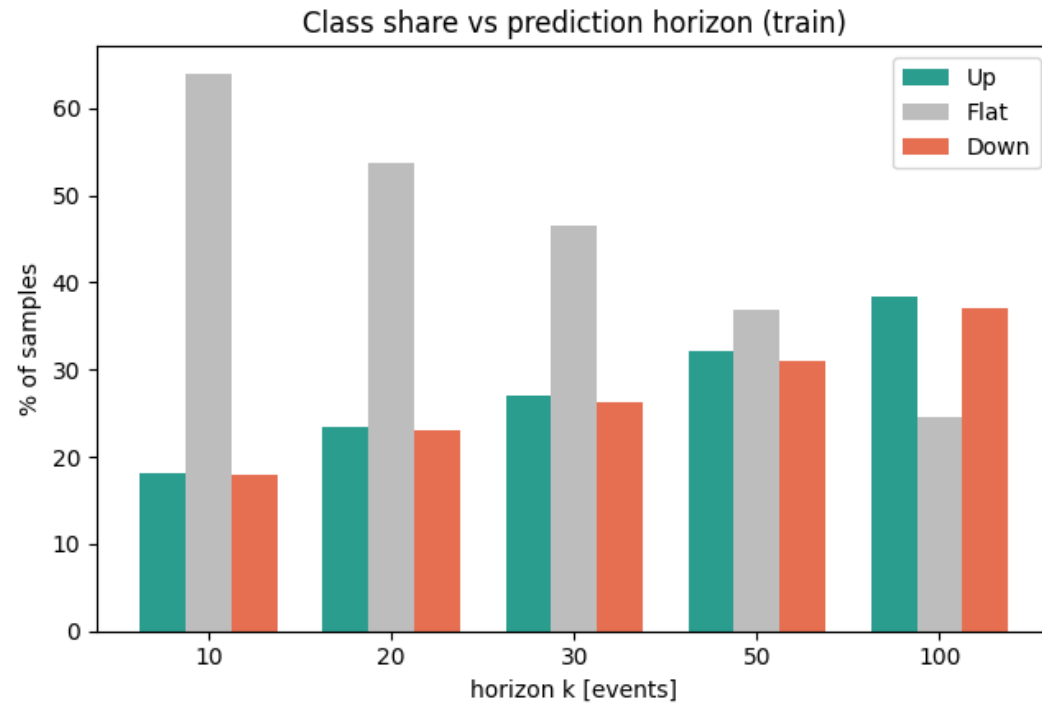
side	price	volume
↑ L4–L10 — higher ask prices, further from mid		
ask 3	10.07	800
ask 2	10.06	450
ask 1 best	10.05	200
----- mid = 10.04 -----		
bid 1 best	10.03	300
bid 2	10.02	600
bid 3	10.01	150
↓ L4–L10 — lower bid prices, further from mid		

level i → 4 features

- ask price
- ask volume
- bid price
- bid volume

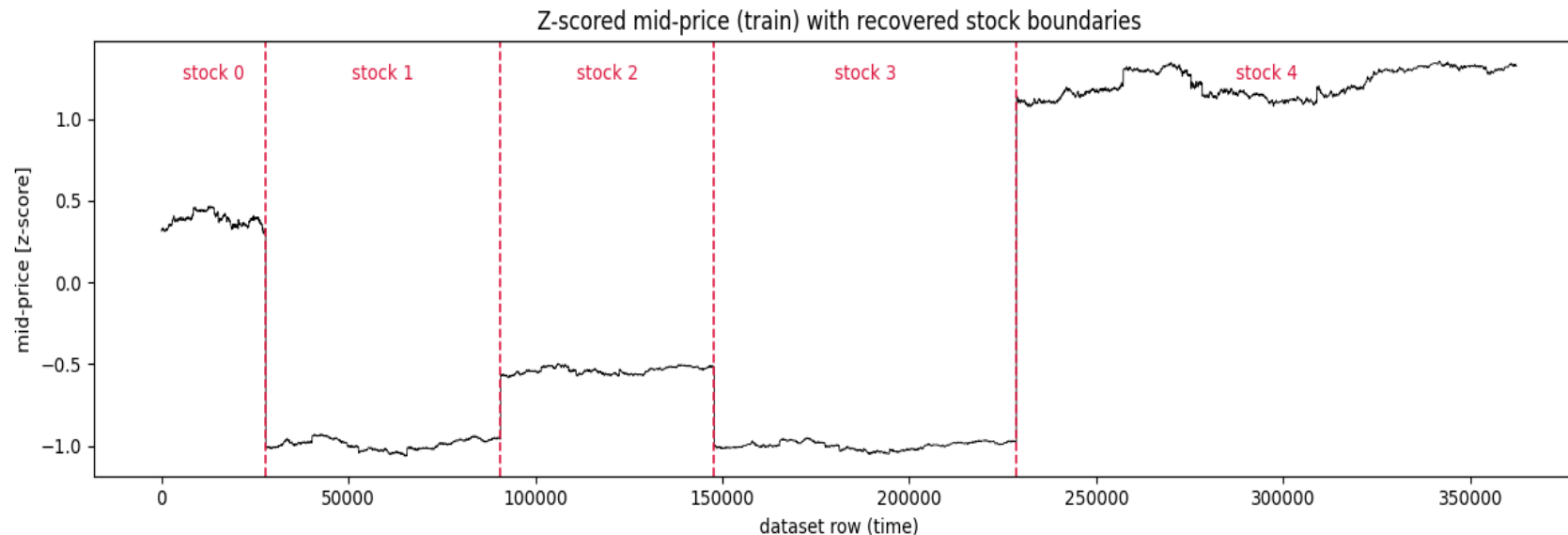
10 × 4 = 40 features

Class imbalance



Data split

- The original data predefines train and test set
- We carve out our own validation set from the train set
- We take the **last 20%**
- Any problems?



Stock segments

- **Problem:** FI-2010 concatenates 5 stocks into one file with no boundary markers.
- temporal 80/20 split → only stock 4 in the validation set (37% of rows)
- **Fix:** recover boundaries as the **4 largest L1 ask-price jumps** (4-50 times bigger than the 5th jump across train/test).

	rank	row index	delta ask L1	scale vs rank 10
0	1	228623	2.0932	50.9x
1	2	27856	1.3338	32.4x
2	3	147839	0.4910	11.9x
3	4	90663	0.3910	9.5x
4	5	257241	0.0931	2.3x
5	6	309048	0.0921	2.2x

Hardcase definition

Why bother defining hard cases at all?

- **The Illusion:** High average accuracy is misleading because the market is quiet 70% of the time.
- **The Target:** "Hard Cases" are the chaotic split-seconds when prices actually change direction.
- **The Goal:** Isolate these extreme moments to stress-test our lightweight model under real-world chaos.

Filter 1: Volume Shock (Iceberg Order Signature)

Concept: Captures massive hidden trades ("Iceberg Orders") executing quietly right before a price explosion.

Price Condition: Local volatility over 20 ticks approaches zero (price is completely frozen).

Volume Condition: Level 1 Volume Imbalance explodes (huge supply/demand wall forms).

Result: Market breaks out violently, causing lightweight models to experience a predictive collapse.



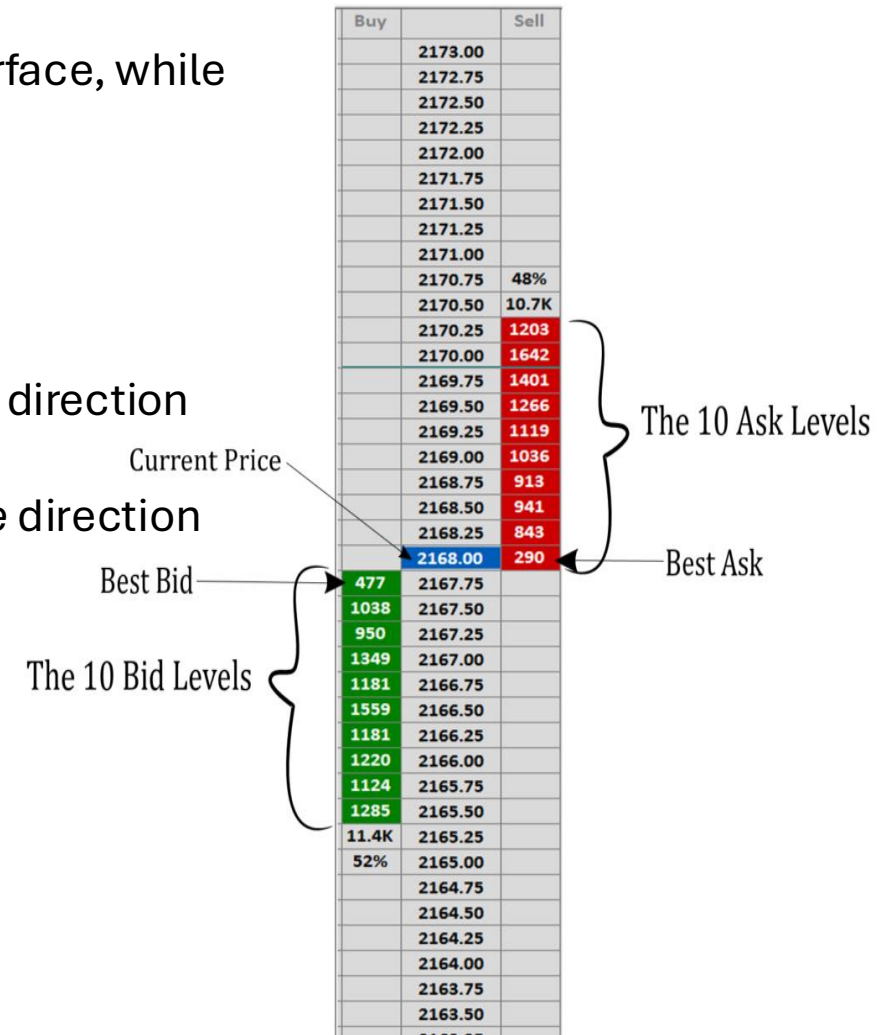
Filter 2: Depth Divergence (Order Book Contradiction)

Concept

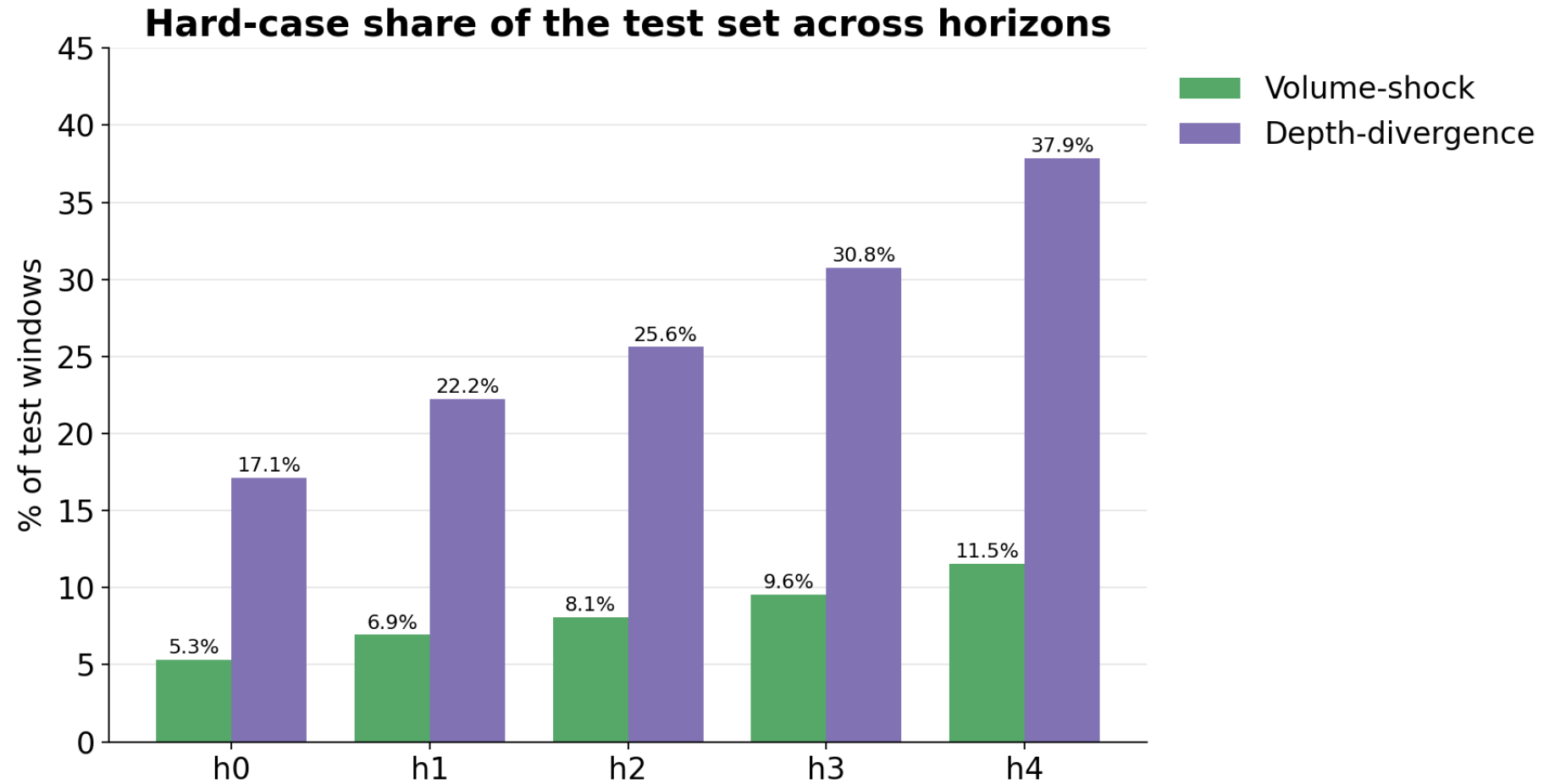
- **"Market Baiting"**: Fake immediate pressure is created on the surface, while the real institutional intent is hidden deep inside the book.
- **The Trap**: Looking only at the surface level leads to a complete misinterpretation of where the market is actually going.

The Mathematical Trigger

- **Surface Layer (L_1)**: Directional imbalance points heavily in one direction (e.g., immediate buying pressure).
- **Deep Core (L_5)**: Volume imbalance points in the *exact opposite* direction (e.g., massive institutional selling walls).



Hard-case share



Architecture

- **1. Analytical Baseline**

- **What it does:** A simple mathematical rule looking only at the latest market snapshot.
- **The Logic:** Compares buyers vs. sellers on Level 1 to guess the immediate direction.
- **Role:** Serves as our absolute minimum benchmark.

- **2. The Teacher Model**

- **What it does:** A heavy, neural network with many parameters.
- **The Logic:** Uses deep spatial (2D-CNN) and temporal (LSTM) layers to read the entire order book grid.
- **Role:** Acts as our source of intelligence, capturing complex market anomalies.

- **3. The Student Model**

- **What it does:** A lightweight, high-performance MLP.
- **The Logic:** Flattens the input data to pass it through a few fast, linear layers.
- **Role:** Optimized strictly for speed to meet execution constraints.

Queue Imbalance

Core Logic

Tracks immediate supply and demand using only the latest market snapshot ($t = -1$).

Imbalance Formula:

$$(\text{Bid Volume} - \text{Ask Volume}) / (\text{Bid Volume} + \text{Ask Volume})$$

Decision Boundaries (Threshold = 0.1)

- **Class 0 (Up):** Index is greater than +0.1 -> Buyers dominate the surface layer.
- **Class 2 (Down):** Index is less than -0.1 -> Sellers dominate the surface layer.
- **Class 1 (Flat):** Index stays between -0.1 and +0.1 -> Stationary market state.

Inherent Limitations

- **Zero Memory:** Ignores the rolling lookback history window entirely (uses only 1 tick instead of 50).
- **Surface Blindness:** Looks strictly at Level 1, leaving it completely vulnerable to deep-book manipulation.

Baseline results

Analytical Baseline (Queue Imbalance) – h0 (k=10), all stocks

Overall Accuracy: 25.40%

	Precision	Recall	F1-Score	Support
Up	0.16	0.32	0.21	5,496
Flat	0.64	0.19	0.29	21,180
Down	0.16	0.46	0.24	5,016
Macro Avg	0.32	0.32	0.25	31,692

Knowledge Distillation & Loss Functions

- **The Core Concept**

- **Knowledge Transfer:** Teacher transfers its intelligence to Student.
- **Soft Targets:** The Student learns to mimic the Teacher's exact probability distributions, not just the hard labels.

- **Teacher Loss**

- **Cross-Entropy Loss:** Measures errors directly against the market Ground Truth.
- *Goal:* Maximize raw accuracy during initial training.

- **Student Loss**

- **1. Student Cross-Entropy:** Keeps the model aligned with the real market Ground Truth.
- **2. KL Divergence:** Penalizes the Student whenever its output drifts away from the Teacher's smooth predictions.
- *Goal:* Force the simple MLP to absorb the spatial and temporal insights of the heavy network.

The Teacher Model

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    # Input shape context: (Batch, Window_Size, Features) -> (B, T, 40)
    # Add channel dimension for Conv2d processing -> (B, 1, T, 40)
    x = x.unsqueeze(1)

    x = self.conv1(x) # Out: (B, 16, T, 20)
    x = self.conv2(x) # Out: (B, 16, T, 10)
    x = self.conv3(x) # Out: (B, 32, T, 1)
    # conv4 has kernel (4, 1) with padding (1, 0), so the temporal axis
    # shrinks from T to T-1 (e.g. 10 -> 9).
    x = self.conv4(x) # Out: (B, 32, T-1, 1)

    # Reshape spatial context tensor maps for LSTM sequence compliance
    x = x.squeeze(3) # Out: (B, 32, T-1)
    x = x.transpose(1, 2) # Out: (B, T-1, 32)

    # Forward sequence execution through LSTM layers
    lstm_out, _ = self.lstm(x)

    # Take the hidden representation from the last historical step (t = -1)
    out_space = lstm_out[:, -1, :]

    return self.fc(out_space)
```

```
# Convolutional block 1: Spatial features extraction from the LOB rows
self.conv1 = nn.Sequential(
    nn.Conv2d(in_channels=1, out_channels=16, kernel_size=(1, 2), stride=(1, 2)),
    nn.BatchNorm2d(16),
    nn.LeakyReLU(negative_slope=0.01)
)

# Convolutional block 2: Further rows abstraction
self.conv2 = nn.Sequential(
    nn.Conv2d(in_channels=16, out_channels=16, kernel_size=(1, 2), stride=(1, 2)),
    nn.BatchNorm2d(16),
    nn.LeakyReLU(negative_slope=0.01)
)

# Convolutional block 3: Mapping across the remaining depth levels (10 levels total)
self.conv3 = nn.Sequential(
    nn.Conv2d(in_channels=16, out_channels=32, kernel_size=(1, 10)),
    nn.BatchNorm2d(32),
    nn.LeakyReLU(negative_slope=0.01)
)

# Convolutional block 4: Micro-temporal modeling inside the window context
self.conv4 = nn.Sequential(
    nn.Conv2d(in_channels=32, out_channels=32, kernel_size=(4, 1), padding=(1, 0)),
    nn.BatchNorm2d(32),
    nn.LeakyReLU(negative_slope=0.01)
)

# Recurrent layer processing aggregated temporal features
self.lstm = nn.LSTM(input_size=32, hidden_size=64, num_layers=1, batch_first=True)

# Final classification head matching the 3 target classes
self.fc = nn.Linear(64, num_classes)
```


The Student Model

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    # Input shape x: (Batch, Window_Size, Features) -> (B, T, 40)
    # Flatten the sequence dimensions to (Batch, T * 40)
    x = x.view(x.size(0), -1)

    # Output logits for the 3 target classes (Up, Flat, Down)
    return self.mlp(x)
```

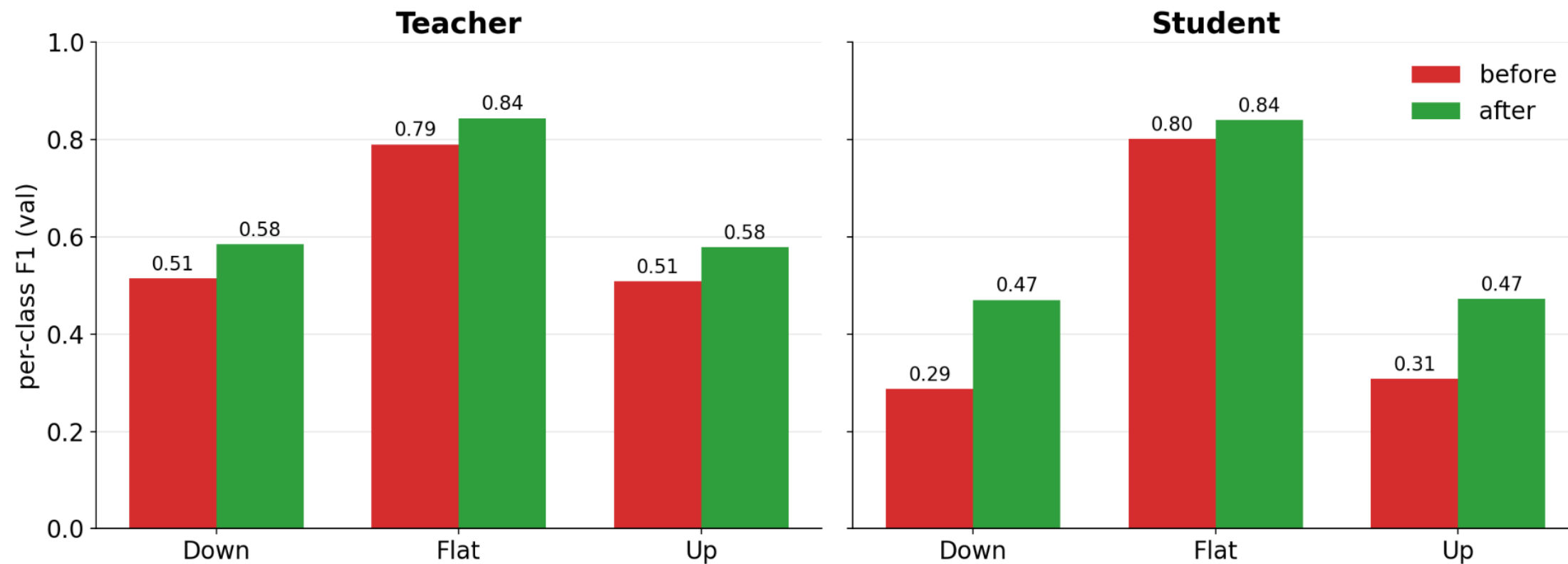
```
# Calculate the total input dimension after flattening (T * 40)
input_dim = window_size * num_features

# Shallow, high-performance architecture
self.mlp = nn.Sequential(
    nn.Linear(input_dim, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Dropout(0.2),

    nn.Linear(128, 64),
    nn.BatchNorm1d(64),
    nn.ReLU(),

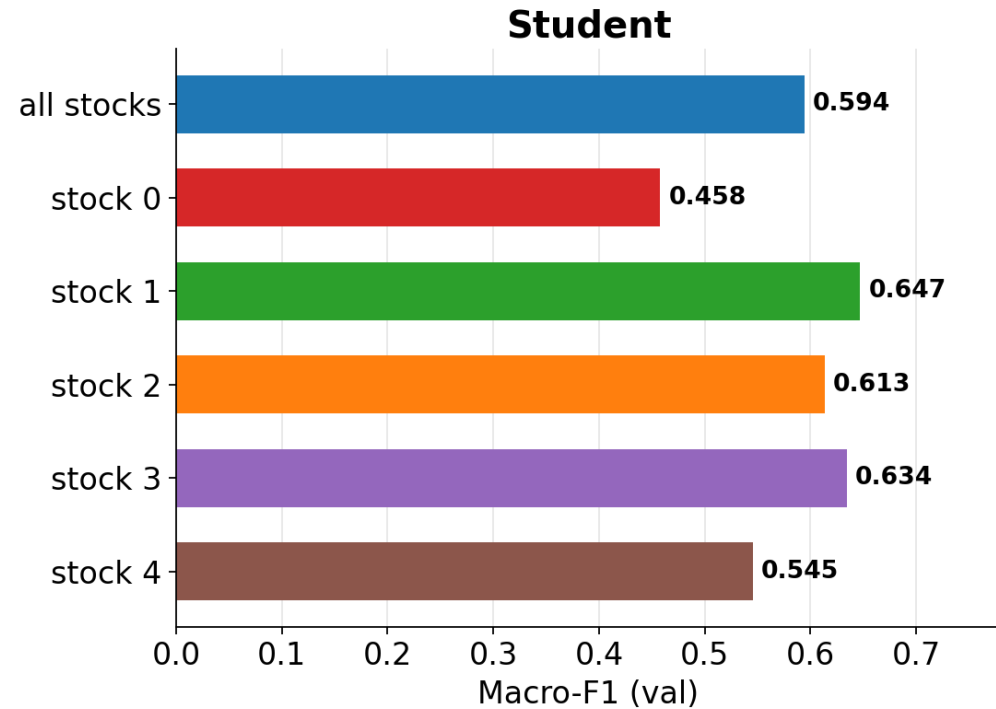
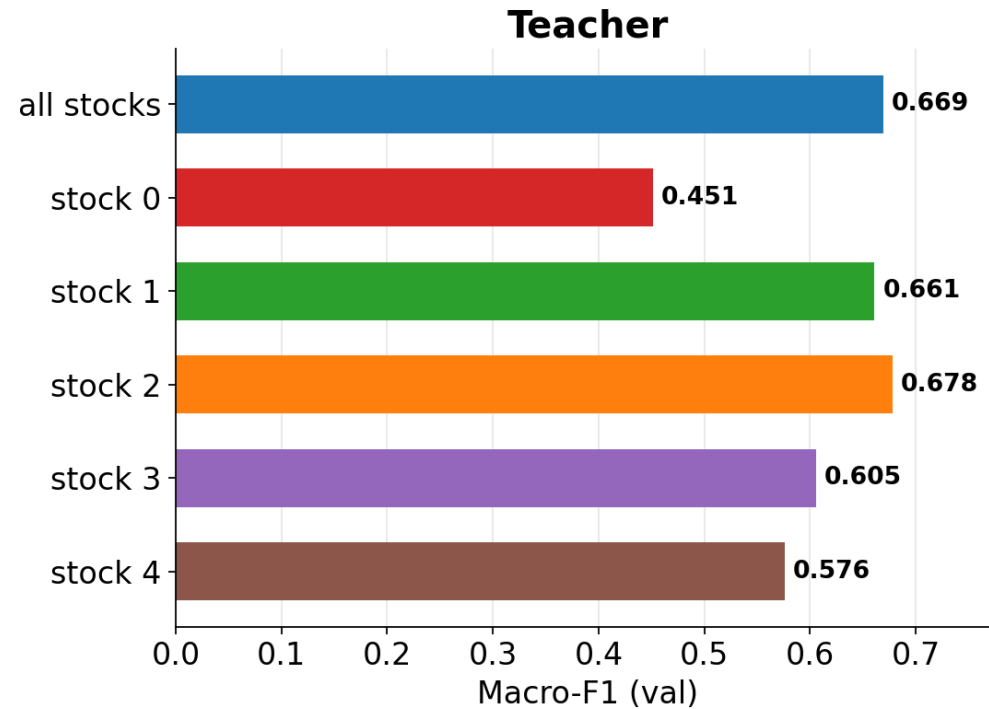
    nn.Linear(64, num_classes)
)
```

Per-stock data split impact



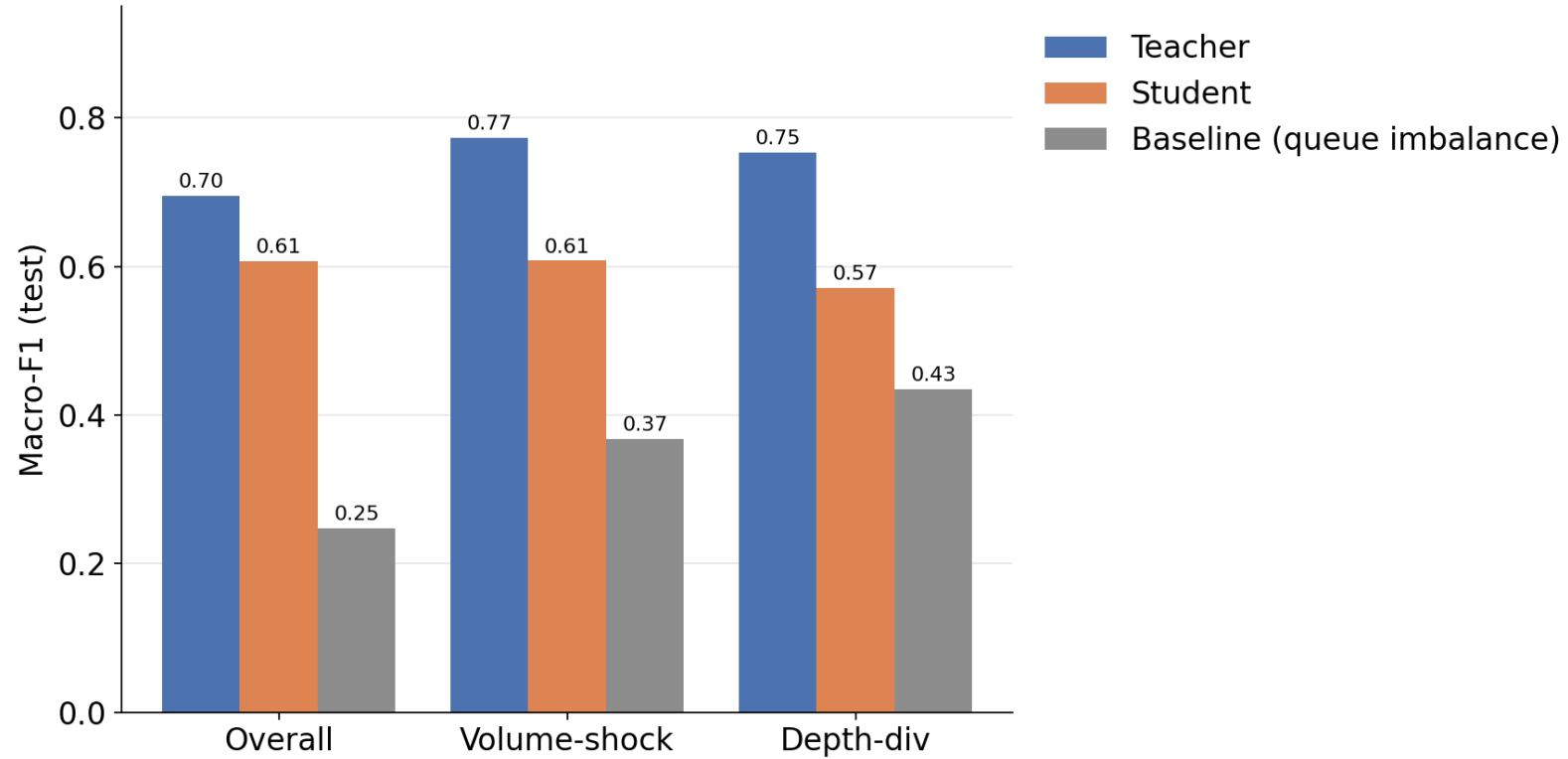
Per-stock training

Per-stock training: Macro-F1 by stock



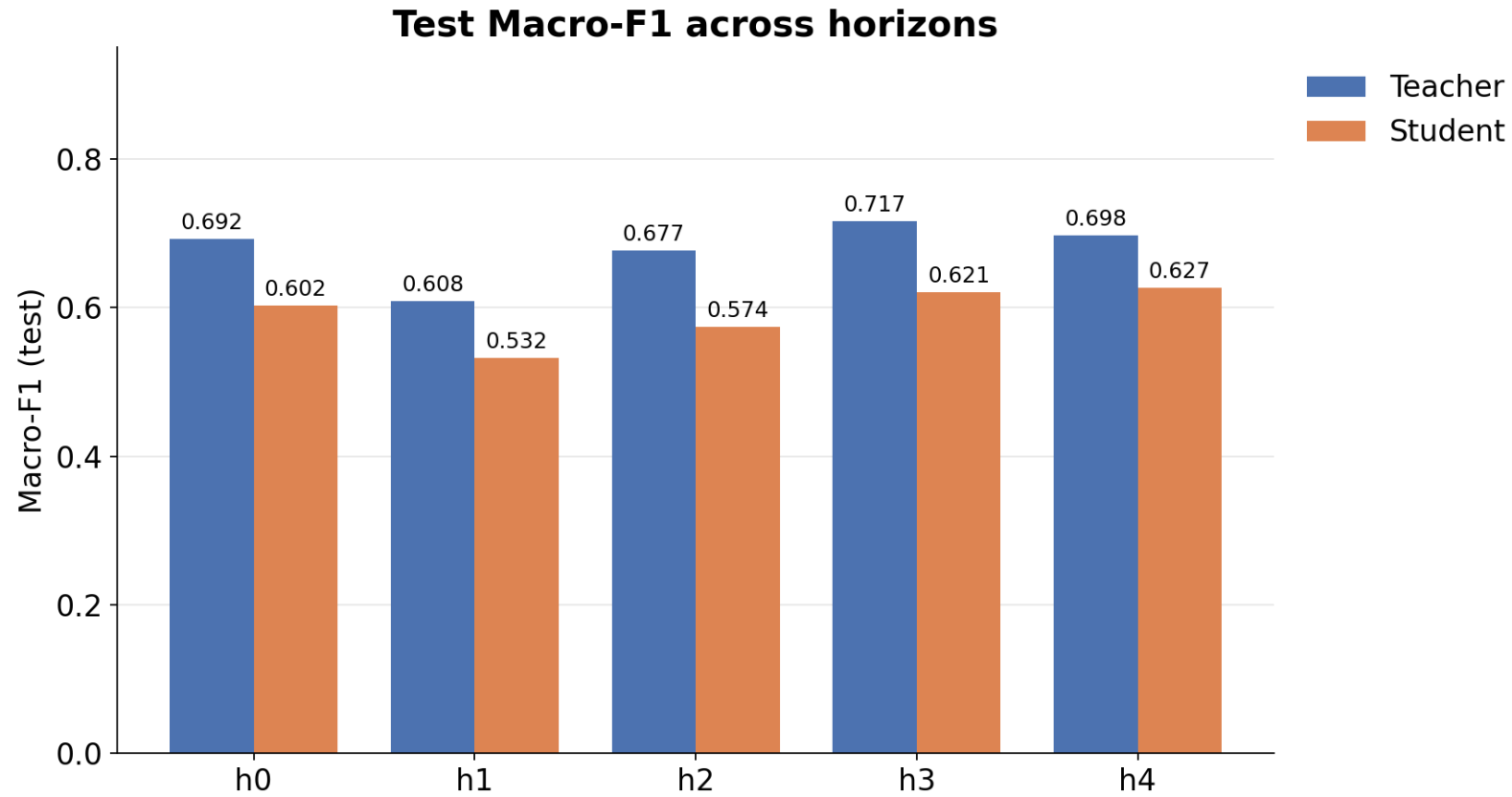
Final results

Test Macro-F1 — Teacher vs Student vs Baseline (h0)

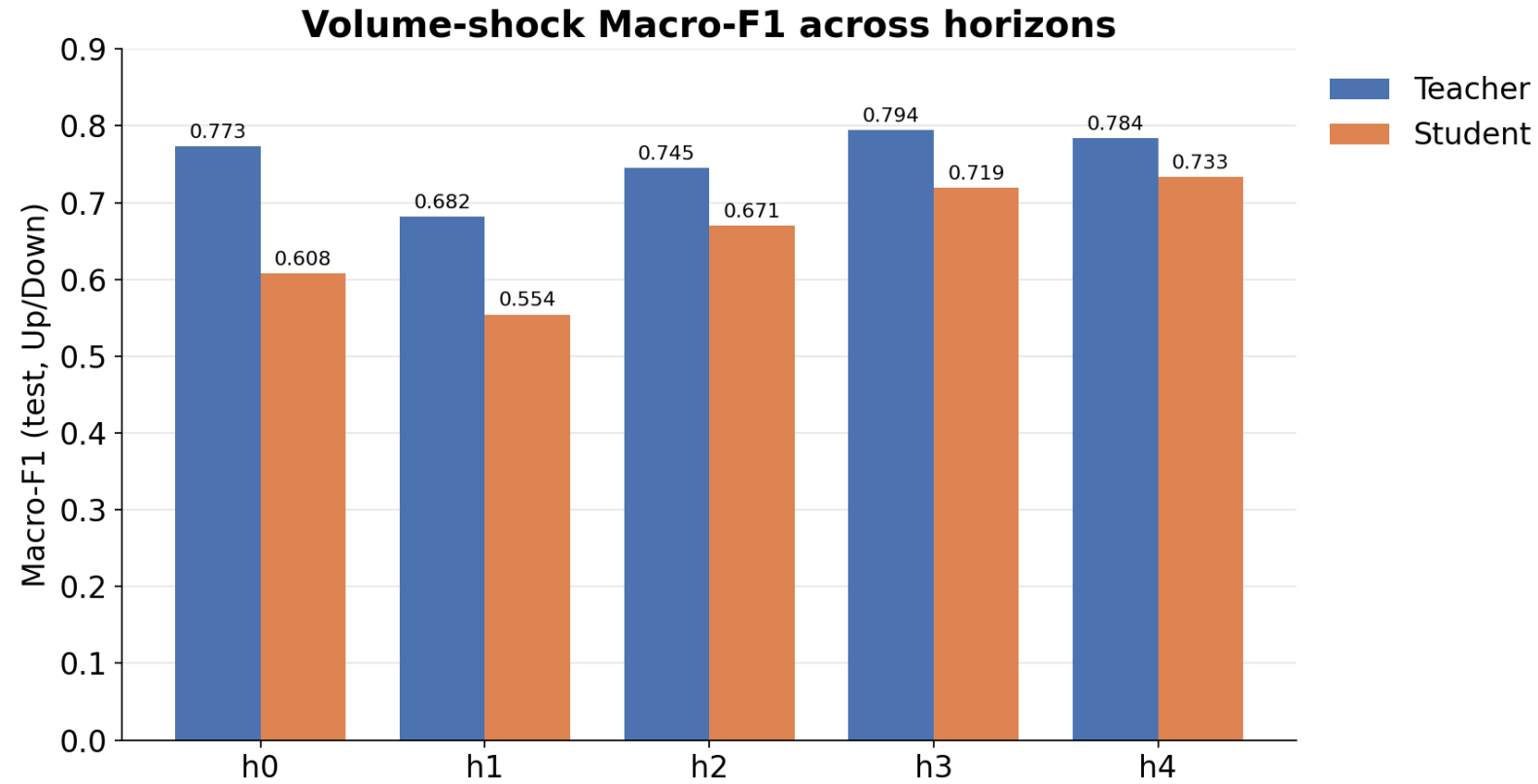


hard cases (Volume-shock, Depth-div): macro-F1 over Up/Down only

Final results across horizons

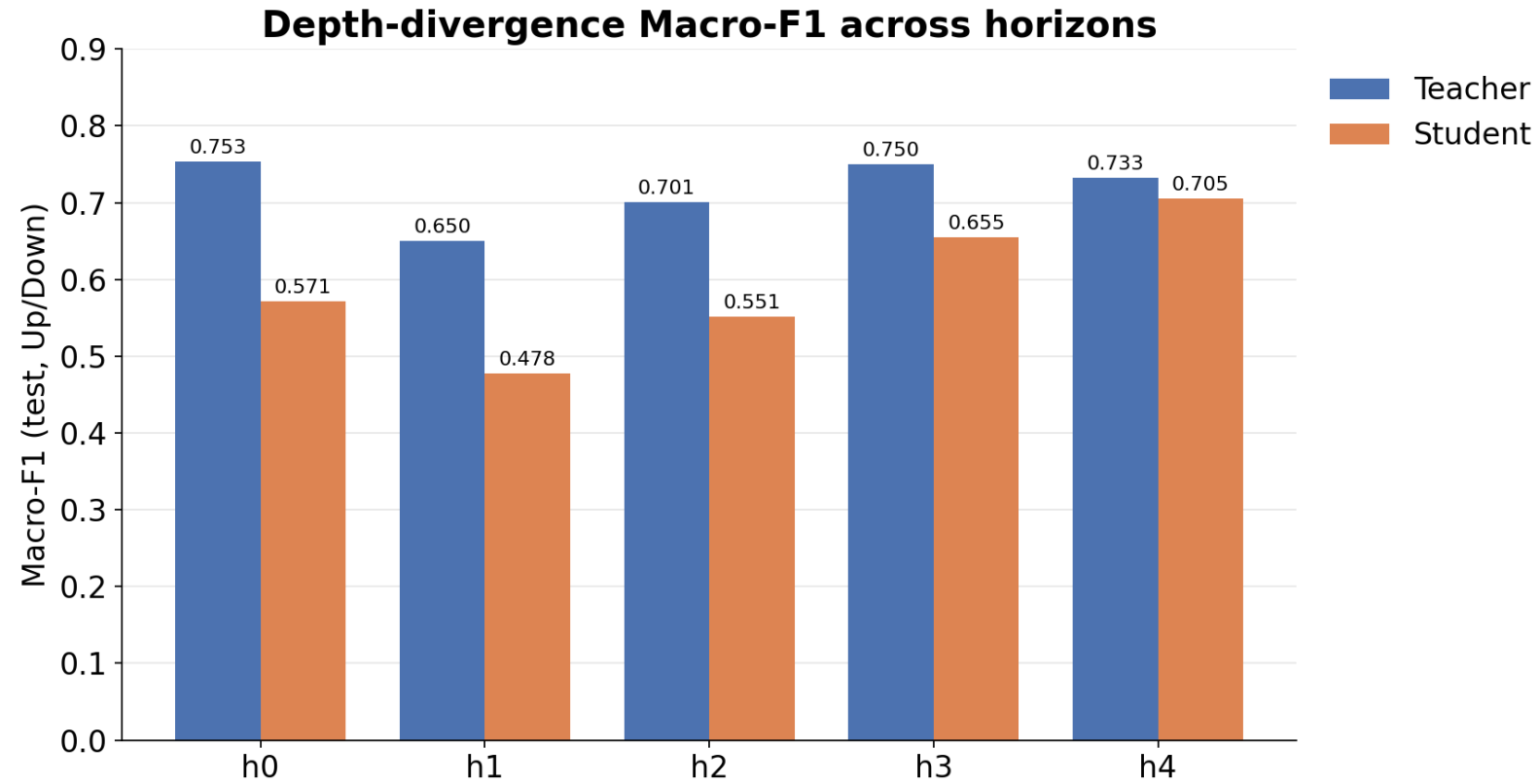


Final results: volume shock



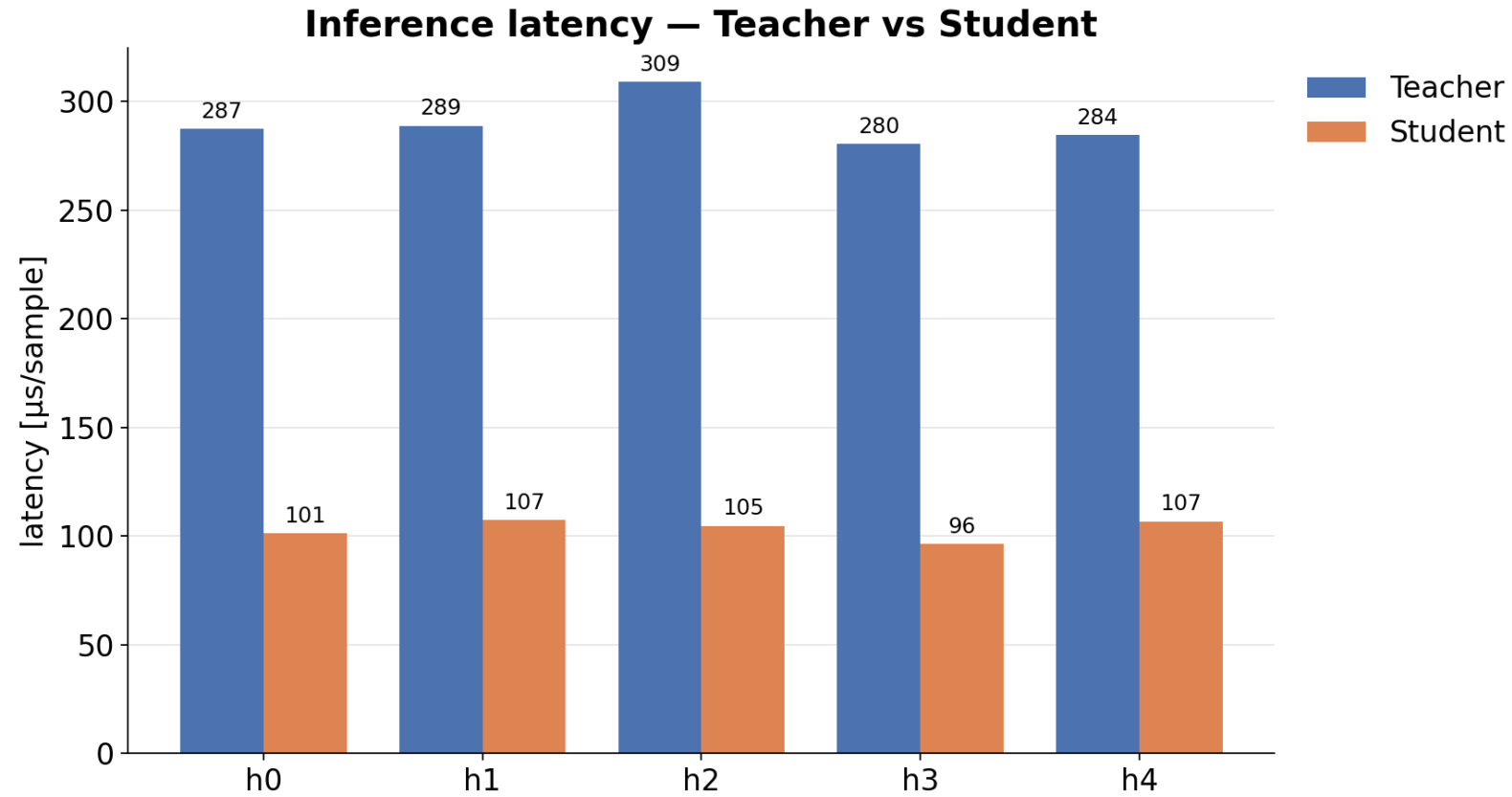
hard-case macro-F1 computed over directional classes only (Up / Down)

Final results: depth divergence



hard-case macro-F1 computed over directional classes only (Up / Down)

Final results: inference latency



Thanks for listening!